SE Lab 1

S Prithika
PES2UG24CS415
Section - G

1. **Requirements table**

REQ ID	TYPE	DESCRIPTION	PRIORITY	ACCEPTANCE CRITERIA	RATIONALE
FR-001	Functional	The system shall evaluate user targeting rules and return accurate boolean feature flag states.	high	Pass: Target 10% rollout cohort receives enabled flag. Fail: User outside cohort receives enabled flag.	Core evaluation logic.
FR-002	Functional	The system shall allow a software engineer to create a new feature flag with a unique key, description, and a default (off) state	high	When a flag key that does not already exist is submitted, the flag is created with state=OFF and appears in the flag list within 2 seconds.	A flag cannot be evaluated or rolled out until it exists
FR-003	Functional	The system shall support percentage-based cohort rollout, deterministically assigning a given user ID to the same bucket (in/out) on every evaluation for a fixed rollout percentage	high	For a flag set to 25% rollout, evaluating the same user ID 100 times returns the same boolean result all 100 times, and across 10,000 distinct user IDs the enabled proportion is 25% ± 2%.	Sticky, deterministic bucketing is required so a user's experience doesn't flicker between requests.
FR-004	Functional	The system shall allow a release manager to define an environment-specific override (dev/staging/prod) that takes precedence over the default targeting rules for that flag	medium	When a prod override of TRUE is set on a flag whose default targeting would return FALSE, a prod evaluation request returns TRUE	Release Managers need to force a flag on/off in a specific environment for testing or emergency kill-switches
FR-005	Functional	The system shall push updated flag states to connected Client SDK instances in real time, without requiring the client to poll.	high	When a flag state is changed, all currently-connected SDK instances receive the updated state within 3 seconds, verified via a	Real-time sync is a stated platform capability; stale flags on clients can cause incorrect behaviour in production.

				subscription/webhook event log.	
NFR-001	Non-functional	The feature flag evaluation API must execute in under 15 ms to minimize overhead when integrated into client critical paths.	high	Pass: Benchmarking tests confirm target latency and security standards under simulated peak load.	flag checks sit on hot paths so latency directly impacts app performance.
NFR-002	Non-functional	The system shall enforce role-based access control (RBAC) so that only users with the release manager role can approve or modify flag state in the production environment, and every state change shall be written to an immutable audit log.	high	Pass: A Software Engineer account attempting to change a prod flag receives HTTP 403 and no state change is recorded. Every successful change appears in the audit log with actor, timestamp, and old/new value within 1 second.	Prevents unauthorized or accidental production changes and provides traceability for incident review.

2. UML Use-case diagram



3. Use case flow document

Use Case: Evaluate Feature Flag

Actors: Client SDK, System (Feature Flag Service)

Preconditions: The feature flag has been created and has targeting rules or a rollout percentage configured. The Client SDK holds a valid API key for the requested environment.

Postconditions: A boolean flag state has been returned to the Client SDK. The evaluation event has been recorded in the evaluation log. No flag configuration is modified.

Main Success Scenario:

1. Client SDK sends an evaluation request for a flag key, including the user ID and environment.
2. System authenticates the request using the client's environment API key and retrieves the flag's configuration.
3. System computes the user's rollout bucket by hashing the user ID against the flag's configured rollout percentage.
4. System evaluates the targeting rules using the cohort membership and bucket result.
5. System checks whether an environment-specific override has been set for this flag.
6. If no override exists, system keeps the targeting-rule result as the final flag state.
7. System logs the evaluation event, including the flag ID, hashed user ID, returned state, and timestamp.
8. System returns the boolean flag state to the Client SDK.
9. Use case ends successfully.

Alternate Flow:

5a. Environment Override Present

5a1. System finds an active environment-specific override configured by the Release Manager for this flag.

5a2. System discards the targeting-rule result and uses the override value as the final flag state instead.

5a3. Flow resumes at step 7 (log the evaluation event, noting override_applied = true) and continues to step 8.